Pizza Store Application

Capstone 2

Version 7.1 Y

Setup

Capstone Setup

For this capstone you should create a public GitHub repo for this Capstone project. Make sure that the capstone repo has a meaningful name, like `PIZZA-licious` (do not just give it a generic name like `capstone-2`)

Clone the `PIZZA-licious` repo into the `C:/pluralsight/capstones` folder.

Create a GitHub project board to manage your work. Use the project requirements to create user stories on your board so that you can manage your time throughout the project.

Be sure to read all project requirements before beginning to code.

Capstone Description

This project is the point of sales application for PIZZA-licious, a custom pizza shop. Currently at PIZZA-licious our customers can fully customize their pizza orders. Until recently we have been managing all orders in person and are currently taking all orders on paper. But our business is growing, and we need a way to automate the order process (and eventually even make it available online).

This application should take full advantage of Object Oriented Analysis and Design. In other words, you should make use of OOP concepts throughout this process.

Create classes and interfaces as necessary to build this solution. As you begin this project you should start by creating a class diagram of the classes, interfaces that you will need. This will be your starting point, and it may change as your application progresses throughout the week. You should keep your diagram current. You should also save the diagram to your repository.

HINT: Use the requirements below to look for nouns and verbs to help you create your class diagram.

Application Requirements

The pizza is the core of our business. Customers can order pizzas in 3 sizes (Personal 8", Medium 12", Large 16"). When they order, they choose the type of crust that they would like (thin, regular, thick, or cauliflower). The customer can then choose their toppings. Toppings are categorized as regular and premium. Meats and cheeses are premium toppings, but most other toppings are considered regular.

Customers can request for extra toppings, but premium toppings come at an additional cost. Each pizza can have stuffed crust.

A customer can place an order with 0 or more pizzas on the order. If a customer places an order with 0 pizzas, they must purchase garlic knots or a drink.

When a customer places the order, they should be prompted to customize each pizza one at a time.

A customer should also be able to add drinks and garlic knots to their order.

When they have completed their order, the application should display the order details, including the list of pizzas that were ordered with all the toppings so that the customer can verify that the order is correct. The screen should also display the total cost of the order.

When the customer completes the order, the order details should be saved to a receipts folder. Each order should have its own receipt file, and it should be named by the date and time that the order was placed

(`yyyyMMdd-hhmmss.txt` - i.e. `20230329-121523.txt`)

Pizza Prices	**8”**	**12”**	**16”**
Crust - thin - regular - thick - cauliflower	8.50	12.00	16.50
Toppings	**8”**	**12”**	**16”**
Meats - pepperoni - sausage - ham - bacon - chicken - meatball	1.00	2.00	3.00
Extra Meat	.50	1.00	1.50
Cheese - Mozzarella - Parmesan - Ricotta - Goat Cheese - Buffalo	.75	1.50	2.25
Extra Cheese	.30	.60	.90
Regular Toppings - onions - mushrooms - bell peppers - olives - tomatoes - spinach - basil - pineapple - anchovies	Included	Included	included
Sauces - marinara - alfredo - pesto - bbq - buffalo - olive oil	Included	Included	Included
Sides - red pepper	Included	Included	Included

- parmesan			
Other Products	**Small**	**Medium**	**Large**
Drinks	2.00	2.50	3.00
Garlic Knots	1.50		

Your application must include several screens with the listed features to be considered complete:

- **Home Screen**
  - The home screen should give the user the following options. The application should continue to run until the user chooses to exit.
    - **1) New Order**
    - **0) Exit** - exit the application
- **Order Screen** - All entries should show the newest entries first
  - **1) Add Pizza**
  - **2) Add Drink**
  - **3) Add Garlic Knots**
  - **4) Checkout**
  - **0) Cancel Order** - delete the order and go back to the home page
- **Add Pizza** - the add pizza screen will walk the user through several options to create the pizza
  - **Select your type:**
  - **Pizza size:**
  - **Toppings:** - the user should be able to add extras of each topping
    - **Meat:**
    - **Cheese:**
    - **Other toppings:**
    - **Select sauces:**
  - **Would you like the pizza with stuffed crust?**
- **Add Drink** - select drink size and flavor
- **Add Garlic Knots**
- **Checkout** - display the order details and the price
  - **Confirm** - create the receipt file and go back to the home screen
  - **Cancel** - delete order and go back to the home screen

(Optional Bonus) Challenge Yourself

If you have time and want to challenge yourself, consider the following:

Create several Signature Pizzas that are a template for what the customer can order. The customer could then customize the existing toppings to either remove or add more toppings to the pizza.

HINT: Signature Pizzas could be managed by creating custom classes that inherit from the Pizza class.

Signature Pizzas might include:

- **Margherita**
  - 12”
  - Regular
  - Mozzarella
  - Tomatoes
  - Basil
  - Marinara
  - Olive Oil
- **Veggie**
  - 8”
  - Regular
  - Bell Peppers
  - Spinach
  - Olives
  - Onions
  - Marinara
  - Mozzarella

Other General Project Requirements

Your project must also meet the following requirements:

Repository

- Your code must be in a public GitHub repository
- You must use Git branches appropriately
- The repository must contain an appropriate Git commit history
    - **Minimally**, you should have a commit for each meaningful piece of work completed
- It must contain an informative README file that:
    - Describes your project
- Include exports or screenshots of your diagrams

Class Demonstrations

Each student will be given 10 minutes to demonstrate their project to the class on "project demonstration day". During this time, you will:

- Present your application - run through the different screens and scenarios
- Describe / show one interesting piece of code that you wrote
- Answer questions from the audience if time permits